

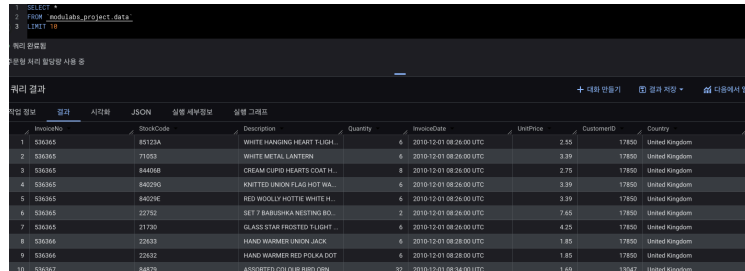
고객을 세그멘테이션하자 [프로젝트] - 전우진

11-2. 데이터 불러오기

데이터 살펴보기

- 테이블에 있는 10개의 행만 출력하기

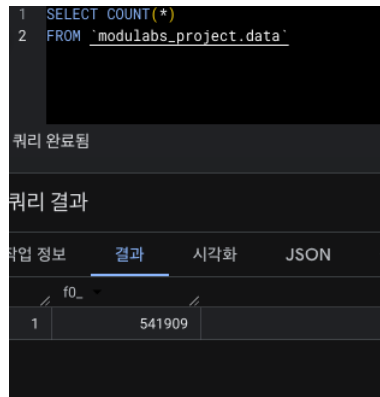
```
SELECT *  
FROM `modulabs_project.data`  
LIMIT 10
```



	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	536365	85123A	WHITE HANGING HEART T-LIGHT...	6	2010-12-01 08:26:00 UTC	2.55	17850	United Kingdom
2	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
3	536365	844068	CREAM CLIPD HEARTS COAT H...	8	2010-12-01 08:26:00 UTC	2.75	17850	United Kingdom
4	536365	842045	KNITTED UNION JACK HOT WA...	6	2010-12-01 08:26:00 UTC	5.39	17850	United Kingdom
5	536365	842046	RED WOOLLY APOSTLE WHITE SL...	4	2010-12-01 08:26:00 UTC	5.39	17850	United Kingdom
6	536365	22752	SET 7 BABUSHKA NESTING BO...	2	2010-12-01 08:26:00 UTC	7.65	17850	United Kingdom
7	536365	21720	GLASS STAR FROSTED T-LIGHT...	6	2010-12-01 08:26:00 UTC	4.25	17850	United Kingdom
8	536366	22633	HAND WARMER UNION JACK	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom
9	536366	22632	HAND WARMER RED POLKA DOT	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom
10	536367	844069	KNITTED UNION JACK HOT WA...	6	2010-12-01 08:28:00 UTC	4.65	17850	United Kingdom

- 전체 데이터는 몇 행으로 구성되어 있는지 확인하기

```
SELECT COUNT(*)  
FROM `modulabs_project.data`
```



	f0_
1	541909

데이터 수 세기

```
SELECT  
COUNT(InvoiceNo) AS COUNT_InvoiceNo,  
COUNT(StockCode) AS COUNT_StockCode,  
COUNT(Description) AS COUNT_Description,  
COUNT(Quantity) AS COUNT_Quantity,  
COUNT(InvoiceDate) AS COUNT_InvoiceDate,  
COUNT(UnitPrice) AS COUNT_UnitPrice,  
COUNT(CustomerID) AS COUNT_CustomerID,  
COUNT(Country) AS COUNT_Country  
FROM `modulabs_project.data`
```

- COUNT 함수를 사용해서, 각 컬럼별 데이터 포인트의 수를 세어 보기

```

1 SELECT
2     COUNT(InvoiceNo) AS COUNT_InvoiceNo,
3     COUNT(StockCode) AS COUNT_StockCode,
4     COUNT(Description) AS COUNT_Description,
5     COUNT(Quantity) AS COUNT_Quantity,
6     COUNT(InvoiceDate) AS COUNT_InvoiceDate,
7     COUNT(UnitPrice) AS COUNT_UnitPrice,
8     COUNT(CustomerID) AS COUNT_CustomerID,
9     COUNT(Country) AS COUNT_Country
10 FROM `modulabs_project.data`
11

```

쿼리 완료됨

수문형 처리 할당량 사용 중

쿼리 결과

작업 정보	결과	시각화	JSON	실행 세부정보	실행 그래프			
	COUNT_InvoiceNo	COUNT_StockCode	COUNT_Description	COUNT_Quantity	COUNT_InvoiceDate	COUNT_UnitPrice	COUNT_CustomerID	COUNT_Country
1	541909	541909	540455	541909	541909	541909	406829	541909

11-4. 데이터 전처리 방법(1): 결측치 제거

컬럼 별 누락된 값의 비율 계산

- 각 컬럼 별 누락된 값의 비율을 계산
 - 각 컬럼에 대해서 누락 값을 계산한 후, 계산된 누락 값을 UNION ALL을 통해 합치기

```

SELECT
    'InvoiceNo' AS column_name,
    ROUND(SUM(CASE WHEN InvoiceNo IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `modulabs_project.data`

UNION ALL

SELECT
    'StockCode' AS column_name,
    ROUND(SUM(CASE WHEN StockCode IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `modulabs_project.data`

UNION ALL

SELECT
    'StockCode' AS column_name,
    ROUND(SUM(CASE WHEN StockCode IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `modulabs_project.data`

UNION ALL

SELECT
    'Quantity' AS column_name,
    ROUND(SUM(CASE WHEN Quantity IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `modulabs_project.data`

UNION ALL

SELECT
    'InvoiceDate' AS column_name,
    ROUND(SUM(CASE WHEN InvoiceDate IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `modulabs_project.data`

UNION ALL

SELECT
    'UnitPrice' AS column_name,
    ROUND(SUM(CASE WHEN UnitPrice IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `modulabs_project.data`

```

```

ntage
FROM `modulabs_project.data`

UNION ALL

SELECT
  'CustomerID' AS column_name,
  ROUND(SUM(CASE WHEN CustomerID IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_perce
ntage
FROM `modulabs_project.data`

UNION ALL

SELECT
  'Country' AS column_name,
  ROUND(SUM(CASE WHEN Country IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_percent
age
FROM `modulabs_project.data`

```

행	column_name	missing_percenta...
1	InvoiceNo	0.0
2	StockCode	0.0
3	StockCode	0.0
4	Quantity	0.0
5	InvoiceDate	0.0
6	UnitPrice	0.0
7	CustomerID	24.93
8	Country	0.0

결측치 처리 전략

- `StockCode = '85123A'` 의 `Description` 을 추출하는 쿼리문을 작성하기

```

SELECT DISTINCT Description
FROM `modulabs_project.data`
WHERE StockCode = '85123A'

```

행	Description
1	WHITE HANGING HEART T-LIGH...
2	?
3	wrongly marked carton 22804
4	CREAM HANGING HEART T-LIG...

결측치 처리

- **DELETE** 구문을 사용하며, **WHERE** 절을 통해 데이터를 제거할 조건을 제시

```

DELETE FROM `modulabs_project.data`
WHERE Description IS NULL OR CustomerID IS NULL

```

--처음에 Description만 제거 + OR 추가해서 한번 더 제거

i 이 문으로 data의 행 1,454개가 삭제되었습니다.

i 이 문으로 data의 행 133,626개가 삭제되었습니다.

11-5. 데이터 전처리(2): 중복값 처리

중복값 확인

- 중복된 행의 수를 세어보기
 - 8개의 컬럼에 그룹 함수를 적용한 후, COUNT가 1보다 큰 데이터를 세어보기

```
SELECT COUNT(*)
FROM (SELECT * FROM `modulabs_project.data`
      GROUP BY InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country
      HAVING COUNT(*) > 1
)
```

행	f0_
1	4837

중복값 처리

- 중복값을 제거하는 쿼리문 작성하기
 - CREATE OR REPLACE TABLE 구문을 활용하여 모든 컬럼(*)을 DISTINCT 한 데이터로 업데이트

```
CREATE OR REPLACE TABLE `modulabs_project.data` AS
SELECT DISTINCT * FROM `modulabs_project.data`
```

i 이 문으로 이름이 data인 테이블이 교체되었습니다.

행	f0_
1	401604

11-6. 데이터 전처리(3): 오류값 처리

InvoiceNo 살펴보기

- 고유(unique)한 InvoiceNo의 개수를 출력하기

```
SELECT COUNT(DISTINCT InvoiceNo)
FROM `modulabs_project.data`
```

행	f0_
1	22190

- 고유한 InvoiceNo를 앞에서부터 100개를 출력하기

```
SELECT DISTINCT InvoiceNo
```

```
FROM `modulabs_project.data`
LIMIT 100
```

행	InvoiceNo
1	541431
2	C541433
3	537626
4	542237
5	549222
6	556201
7	562032

- **InvoiceNo** 가 'C'로 시작하는 행을 필터링 할 수 있는 쿼리문을 작성하기 (100행까지만 출력)

```
SELECT *
FROM project_name.modulabs_project.data
WHERE InvoiceNo LIKE 'C%'
LIMIT 100;
```

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	C541433	22146	MEDIUM CERAMIC TOP-STORA...	-74215	2011-03-18 15:17:00 UTC	1.56	12345	United Kingdom
2	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	183.75	12352	Norway
3	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	280.00	12352	Norway
4	C545329	M	Manual	-1	2011-03-01 15:48:00 UTC	276.5	12352	Norway
5	C547388	37488	CERAMIC CAME DESIGN SPOTT...	-12	2011-03-22 16:07:00 UTC	5.49	12352	Norway
6	C547388	84030	PINK HEART SHAPE EGG FRYIN...	-12	2011-03-22 16:07:00 UTC	5.65	12352	Norway
7	C547388	22784	LANTERN CREAM GAZEBO	-3	2011-03-22 16:07:00 UTC	4.95	12352	Norway

- 구매 건 상태가 **Canceled** 인 데이터의 비율(%) - 소수점 첫번째 자리까지

```
SELECT ROUND(SUM(CASE WHEN InvoiceNo LIKE 'C%' THEN 1 ELSE 0 END)/COUNT(*) * 100, 1)
FROM `modulabs_project.data`
```

행	f0_
1	2.2

StockCode 살펴보기

- 고유한 **StockCode** 의 개수를 출력하기

```
SELECT COUNT(DISTINCT StockCode)
FROM `modulabs_project.data`
```

행	f0_
1	3684

- 어떤 제품이 가장 많이 판매되었는지 보기 위하여 **StockCode** 별 등장 빈도를 출력하기
 - 상위 10개의 제품들을 출력하기

```
SELECT StockCode, COUNT(*) AS sell_cnt
FROM `modulabs_project.data`
GROUP BY StockCode
ORDER BY sell_cnt DESC
LIMIT 10
```

행	StockCode	sell_cnt
1	85123A	2065
2	22423	1894
3	85099B	1659
4	47566	1409
5	84879	1405
6	20725	1346
7	22720	1224

- **StockCode**의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고

행	number_count	stock_cnt
1	5	3676
2	0	7
3	1	1

- 숫자가 0~1개인 값들에는 어떤 코드들이 들어가 있는지 출력하기

```
SELECT DISTINCT StockCode, number_count
FROM (
  SELECT StockCode,
    LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
  FROM `modulabs_project.data`
)
WHERE number_count = 1 OR number_count =0
```

행	StockCode	number_count
1	POST	0
2	M	0
3	C2	1
4	D	0
5	BANK CHARGES	0
6	PADS	0
7	DOT	0
8	CRUK	0

- **StockCode**의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고
 - 숫자가 0~1개인 값들을 가지고 있는 데이터 수는 전체 데이터 수 대비 몇 퍼센트인지 구하기 (소수점 두 번째 자리까지)

```
SELECT ROUND(SUM(CASE WHEN StockCode IN ('POST', 'M', 'C2', 'D', 'BANK CHARGES', 'PADS', 'DOT', 'CRUK') THEN 1 ELSE 0 END)/ COUNT(*)*100,2)
FROM (
  SELECT StockCode,
    LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
  FROM `modulabs_project.data`
)
```

행	f0_
1	0.48

- 제품과 관련되지 않은 거래 기록을 제거하기

```
DELETE FROM `modulabs_project.data`
WHERE StockCode IN (
  SELECT DISTINCT StockCode
  FROM (
    SELECT StockCode,
    LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
    FROM `modulabs_project.data`
    WHERE StockCode IN ('POST', 'M', 'C2', 'D', 'BANK CHARGES', 'PADS', 'DOT', 'CRUK')
  )
);
```

i 이 문으로 data의 행 1,915개가 삭제되었습니다.

Description 살펴보기

- 고유한 Description 별 출현 빈도를 계산하고 상위 30개를 출력하기

```
SELECT DISTINCT Description , COUNT(*) AS description_cnt
FROM `modulabs_project.data`
GROUP BY Description
LIMIT 30
```

행	Description	description_cnt
1	MEDIUM CERAMIC TOP STORA...	208
2	MINI PAINT SET VINTAGE	335
3	PINK 3 PIECE POLKADOT CUTL...	103
4	FOUR HOOK WHITE LOVEBIRDS	265
5	AIRLINE BAG VINTAGE JET SET...	96
6	CLEAR DRAWER KNOB ACRYLIC...	331
7	ALARM CLOCK BAKELIKE RED	917
8	PURPLE DRAWERKNOB ACRYLI...	151
9	BATHROOM METAL SIGN	60

- 서비스 관련 정보를 포함하는 행들을 제거하기

```
DELETE
FROM `modulabs_project.data`
WHERE
Description IN('Next Day Carriage','High Resolution Image')
```

i 이 문으로 data의 행 83개가 삭제되었습니다.

- 대소문자를 혼합하고 있는 데이터를 대문자로 표준화 하기

```
CREATE OR REPLACE TABLE modulabs_project.data AS
SELECT
  * EXCEPT (Description),
```

```
UPPER(Description) AS Description
FROM `modulabs_project.data`
```

i 이 문으로 이름이 data인 테이블이 교체되었습니다.

UnitPrice 살펴보기

- UnitPrice 의 최솟값, 최댓값, 평균을 구하기

```
SELECT MIN(UnitPrice) AS min_price, MAX(UnitPrice) AS max_price, AVG(UnitPrice) AS avg_price
FROM `modulabs_project.data`;
```

행	min_price	max_price	avg_price
1	0.0	649.5	2.904956757406...

- 단가가 0원인 거래의 개수, 구매 수량(Quantity)의 최솟값, 최댓값, 평균 구하기

```
SELECT MIN(Quantity) AS min_price, MAX(Quantity) AS max_price, AVG(Quantity) AS avg_price
FROM `modulabs_project.data`
WHERE UnitPrice = 0
```

행	min_price	max_price	avg_price
1	1	12540	420.5151515151...

- UnitPrice = 0 를 제거하고 일관된 데이터셋을 유지하기

```
CREATE OR REPLACE TABLE `modulabs_project.data` AS
SELECT *
FROM modulabs_project.data
WHERE UnitPrice != 0
```

i 이 문으로 이름이 data인 테이블이 교체되었습니다.

11-7. RFM 스코어

Recency

- InvoiceDate 컬럼을 연월일 자료형으로 변경하기

```
SELECT DATE(InvoiceDate) AS InvoiceDay, *
FROM `modulabs_project.data`;
```


행	InvoiceDay	InvoiceNo	StockCode	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Description
1	2011-01-18	541431	23166	74215	2011-01-18 10:01:00 UTC	1.04	12346	United Kingdom	MEDIUM CER
2	2011-01-18	541431	23166	74215	2011-01-18 10:17:00 UTC	1.04	12346	United Kingdom	MEDIUM CER
3	2010-12-07	537626	20782	6	2010-12-07 14:57:00 UTC	5.49	12347	Iceland	CAMOUFLAG
4	2010-12-07	537626	22728	4	2010-12-07 14:57:00 UTC	3.75	12347	Iceland	ALARM CLOC
5	2010-12-07	537626	71477	12	2010-12-07 14:57:00 UTC	3.25	12347	Iceland	COLOUR GLA
6	2010-12-07	537626	22773	12	2010-12-07 14:57:00 UTC	1.25	12347	Iceland	GREEN DRAW
7	2010-12-07	537626	22497	4	2010-12-07 14:57:00 UTC	4.25	12347	Iceland	SET OF 2 TIR
8	2010-12-07	537626	22771	12	2010-12-07 14:57:00 UTC	1.25	12347	Iceland	CLEAR DRAW
9	2010-12-07	537626	84910	6	2010-12-07 14:57:00 UTC	3.75	12347	Iceland	PINK 3 PILL

- 가장 최근 구매 일자를 MAX() 함수로 찾아보기

```
SELECT
    MAX(InvoiceDATE) AS most_recent_date,
    DATE(InvoiceDATE) AS InvoiceDay,
FROM `modulabs_project.data`
GROUP BY InvoiceDay;
```

행	most_recent_date	InvoiceDay
1	2011-01-18 16:52:00 UTC	2011-01-18
2	2010-12-07 17:19:00 UTC	2010-12-07
3	2011-01-26 17:43:00 UTC	2011-01-26
4	2011-04-07 19:27:00 UTC	2011-04-07
5	2011-06-09 20:03:00 UTC	2011-06-09
6	2011-08-02 17:31:00 UTC	2011-08-02
7	2011-10-31 17:13:00 UTC	2011-10-31
8	2011-12-07 17:25:00 UTC	2011-12-07
9	2010-12-16 19:51:00 UTC	2010-12-16
10	2011-01-25 17:33:00 UTC	2011-01-25

- 유저 별로 가장 큰 InvoiceDay를 찾아서 가장 최근 구매일로 저장하기

```
SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
FROM `modulabs_project.data`
GROUP BY CustomerID
ORDER BY InvoiceDay DESC;
```

행	CustomerID	InvoiceDay
1	13069	2011-12-09
2	12526	2011-12-09
3	17389	2011-12-09
4	17364	2011-12-09
5	12518	2011-12-09
6	18102	2011-12-09
7	12985	2011-12-09
8	14422	2011-12-09
9	13777	2011-12-09

- 가장 최근 일자(most_recent_date)와 유저별 마지막 구매일(InvoiceDay)간의 차이를 계산하기

```
SELECT
    CustomerID,
    EXTRACT(DAY FROM MAX(InvoiceDay) OVER () - InvoiceDay) AS recency
FROM (
    SELECT
        CustomerID,
        MAX(DATE(InvoiceDate)) AS InvoiceDay
    FROM `modulabs_project.data`
```

```
GROUP BY CustomerID
);
```

행	CustomerID	recency
1	12843	65
2	12882	9
3	12921	3
4	12947	143
5	12949	30
6	13279	64
7	13297	7
8	13314	1

- 최종 데이터 셋에 필요한 데이터들을 각각 정제해서 이어붙이고 지금까지의 결과를 `user_r` 이라는 이름의 테이블로 저장하기

```
CREATE OR REPLACE TABLE `modulabs_project.user_r` AS

WITH last_purchase AS (
  SELECT
    CustomerID,
    MAX(Date(InvoiceDate)) AS InvoiceDay
  FROM `modulabs_project.data`
  WHERE CustomerID IS NOT NULL
  GROUP BY CustomerID
)
SELECT
  CustomerID,
  EXTRACT(DAY FROM MAX(InvoiceDay) OVER () - InvoiceDay) AS recency
FROM last_purchase;
```

행	CustomerID	recency
1	13069	0
2	16705	0
3	15344	0
4	15910	0
5	16446	0
6	17428	0
7	17754	0
8	17001	0
9	18102	0
10	12680	0
11	12526	0
12	12713	0
13	12433	0
14	12662	0
15	16626	0
16	12748	0

Frequency

- 고객마다 고유한 InvoiceNo의 수를 세어보기

```
SELECT
  CustomerID,
  COUNT(DISTINCT InvoiceNo) AS purchase_cnt
```

```
FROM `modulabs_project.data`
GROUP BY CustomerID
```

행	CustomerID	purchase_cnt
1	12346	2
2	12347	7
3	12348	4
4	12349	1
5	12350	1
6	12352	8
7	12353	1
8	12354	1

- 각 고객 별로 구매한 아이템의 총 수량 더하기

```
SELECT
  CustomerID,
  SUM(DISTINCT Quantity) AS item_cnt
FROM `modulabs_project.data`
GROUP BY CustomerID
```

행	CustomerID	item_cnt
1	12346	0
2	12347	477
3	12348	604
4	12349	147
5	12350	52
6	12352	45
7	12353	20
8	12354	107

- 전체 거래 건수 계산과 구매한 아이템의 총 수량 계산의 결과를 합쳐서 `user_rf` 라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE `modulabs_project.user_rf` AS

WITH purchase_cnt AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT InvoiceNo) AS purchase_cnt
  FROM `modulabs_project.data`
  WHERE CustomerID IS NOT NULL
  GROUP BY CustomerID
),
item_cnt AS (
  SELECT
    CustomerID,
    SUM(Quantity) AS item_cnt
  FROM `modulabs_project.data`
  WHERE CustomerID IS NOT NULL
  GROUP BY CustomerID
)

SELECT
  pc.CustomerID,
  pc.purchase_cnt,
  ic.item_cnt,
  ur.renecy
```

```
FROM purchase_cnt AS pc
JOIN item_cnt AS ic
  ON pc.CustomerID = ic.CustomerID
JOIN `modulabs_project.user_r` AS ur
  ON pc.CustomerID = ur.CustomerID;
```

행	CustomerID	purchase_cnt	item_cnt	recency
1	12713	1	505	0
2	14569	1	79	1
3	13298	1	96	1
4	15520	1	314	1
5	13436	1	76	1
6	14204	1	72	2
7	15471	1	256	2
8	15195	1	1404	2
9	17914	1	457	3
10	16569	1	93	3
11	14578	1	240	3
12	12650	1	250	3
13	12478	1	233	3
14	15318	1	642	3
15	12442	1	181	3
16	16528	1	171	3
17	15992	1	17	3
18	14219	1	78	4
19	15097	1	170	4

Monetary

- 고객별 총 지출액 계산 (소수점 첫째 자리에서 반올림)

```
SELECT
  CustomerID,
  ROUND(SUM(UnitPrice * Quantity),3) AS user_total
FROM `modulabs_project.data`
GROUP BY CustomerID
```

행	CustomerID	user_total
1	12346	0.0
2	12347	4310.0
3	12348	1437.24
4	12349	1457.55
5	12350	294.4
6	12352	1265.41
7	12353	89.0
8	12354	1070.4

- 고객별 평균 거래 금액 계산
 - 고객별 평균 거래 금액을 구하기 위해 1) `data` 테이블을 `user_rf` 테이블과 조인(LEFT JOIN) 한 후, 2) `purchase_cnt` 로 나누어서 3) `user_rfm` 테이블로 저장하기

```
CREATE OR REPLACE TABLE `modulabs_project.user_rfm` AS
SELECT
```

```

rf.CustomerID,
rf.purchase_cnt,
rf.item_cnt,
rf.recency,
ut.user_total,
ROUND(ut.user_total / rf.purchase_cnt, 2) AS user_average
FROM `modulabs_project.user_rf` rf
LEFT JOIN (
  SELECT
    CustomerID,
    ROUND(SUM(UnitPrice * Quantity), 2) AS user_total
  FROM `modulabs_project.data`
  GROUP BY CustomerID
) ut
ON rf.CustomerID = ut.CustomerID;

```

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average
1	12713	1	505	0	794.55	794.55
2	13298	1	96	1	360.0	360.0
3	13436	1	76	1	196.89	196.89
4	15520	1	314	1	343.5	343.5
5	14569	1	79	1	227.39	227.39
6	15471	1	256	2	454.48	454.48
7	14204	1	72	2	150.61	150.61
8	15195	1	1404	2	3861.0	3861.0
9	12442	1	181	3	144.06	144.06
10	17914	1	457	3	329.36	329.36
11	15318	1	642	3	312.62	312.62
12	16569	1	93	3	124.2	124.2
13	12650	1	250	3	242.44	242.44
14	15992	1	17	3	41.99	41.99
15	16528	1	171	3	244.41	244.41
16	14578	1	240	3	168.63	168.63
17	12478	1	233	3	545.99	545.99
18	16597	1	184	4	90.04	90.04
19	15097	1	170	4	248.08	248.08
20	12367	1	172	4	150.9	150.9

RFM 통합 테이블 출력하기

- 최종 user_rfm 테이블을 출력하기

```

SELECT *
FROM `modulabs_project.user_rfm`

```

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average
1	12713	1	505	0	794.55	794.55
2	13298	1	96	1	360.0	360.0
3	13436	1	76	1	196.89	196.89
4	15520	1	314	1	343.5	343.5
5	14569	1	79	1	227.39	227.39
6	15471	1	256	2	454.48	454.48
7	14204	1	72	2	150.61	150.61
8	15195	1	1404	2	3861.0	3861.0
9	12442	1	181	3	144.06	144.06
10	17914	1	457	3	329.36	329.36

11-8. 추가 Feature 추출

1. 구매하는 제품의 다양성

- 1) 고객 별로 구매한 상품들의 고유한 수를 계산하기
- 2) `user_rfm` 테이블과 결과를 합치기
- 3) `user_data` 라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE `modulabs_project.user_data` AS
WITH unique_products AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT StockCode) AS unique_products
  FROM `modulabs_project.data`
  GROUP BY CustomerID
)
SELECT ur.*, up.* EXCEPT (CustomerID)
FROM `modulabs_project.user_rfm` AS ur
JOIN unique_products AS up
ON ur.CustomerID = up.CustomerID;
```

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_prod...
1	17382	1	24	65	50.4	50.4	1
2	16428	1	-1	81	-2.95	-2.95	1
3	16257	1	1	176	21.95	21.95	1
4	16078	1	16	283	79.2	79.2	1
5	15070	1	36	372	106.2	106.2	1
6	17948	1	144	147	358.56	358.56	1
7	15753	1	144	304	79.2	79.2	1
8	18141	1	-12	360	-35.4	-35.4	1
9	12791	1	96	373	177.6	177.6	1
10	18184	1	60	15	49.8	49.8	1
11	15524	1	4	24	440.0	440.0	1
12	14351	1	12	164	51.0	51.0	1
13	16093	1	20	106	17.0	17.0	1
14	13185	1	12	267	71.4	71.4	1
15	15316	1	100	326	165.0	165.0	1
16	15195	1	1404	2	3861.0	3861.0	1
17	13017	1	48	7	204.0	204.0	1
18	17102	1	2	261	25.5	25.5	1
19	14576	1	12	372	35.4	35.4	1

2. 평균 구매 주기

- 고객들의 쇼핑 패턴을 이해하는 것을 목표 (고객 별 재방문 주기 살펴보기)
 - 균 구매 소요 일수를 계산하고, 그 결과를 `user_data` 에 통합

```
CREATE OR REPLACE TABLE `modulabs_project.user_data` AS
WITH purchase_intervals AS (
  -- (2) 고객 별 구매와 구매 사이의 평균 소요 일수
  SELECT
    CustomerID,
    CASE WHEN ROUND(AVG(interval_), 2) IS NULL THEN 0 ELSE ROUND(AVG(interval_), 2) END AS average_in
  terval
  FROM (
    -- (1) 구매와 구매 사이에 소요된 일수
    SELECT
      CustomerID,
      DATE_DIFF(InvoiceDate, LAG(InvoiceDate) OVER (PARTITION BY CustomerID ORDER BY InvoiceDate), DA
    Y) AS interval_
    FROM
      `modulabs_project.data`
  )
)
```

```

        WHERE CustomerID IS NOT NULL
    )
    GROUP BY CustomerID
)

SELECT u.*, pi.* EXCEPT (CustomerID)
FROM `modulabs_project.user_data` AS u
LEFT JOIN purchase_intervals AS pi
ON u.CustomerID = pi.CustomerID;

```

3. 구매 취소 경향성

- 고객의 취소 패턴 파악하기
 - 1) 취소 빈도(cancel_frequency) : 고객 별로 취소한 거래의 총 횟수
 - 2) 취소 비율(cancel_rate) : 각 고객이 한 모든 거래 중에서 취소를 한 거래의 비율
 - 취소 빈도와 취소 비율을 계산하고 그 결과를 `user_data`에 통합하기
(취소 비율은 소수점 두번째 자리)

```

CREATE OR REPLACE TABLE `modulabs_project.user_data` AS

WITH TransactionInfo AS (
    SELECT
        CustomerID,
        COUNT(DISTINCT InvoiceNo) AS total_transactions,
        SUM(DISTINCT CASE WHEN InvoiceNo LIKE 'C%' THEN 1 ELSE 0 END) AS cancel_frequency
    FROM `modulabs_project.data`
    GROUP BY CustomerID
)

SELECT u.*, t.* EXCEPT(CustomerID), (t.cancel_frequency/t.total_transactions) AS cancel_rate
FROM `modulabs_project.user_data` AS u
LEFT JOIN TransactionInfo AS t
ON u.CustomerID = t.CustomerID;

```

일	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_prod	average_item	total_transac	cancel_freq	cancel_rate
1	17986	1	10	56	20.8	20.8	1	0.0	1	0	0.0
2	17331	1	16	123	175.2	175.2	1	0.0	1	0	0.0
3	16953	1	10	30	20.8	20.8	1	0.0	1	0	0.0
4	16162	1	4	252	37.4	37.4	2	0.0	1	0	0.0
5	13967	1	42	145	80.7	80.7	2	0.0	1	0	0.0
6	14816	1	5	197	271.85	271.85	4	0.0	1	0	0.0
7	18191	1	140	261	207.8	207.8	7	0.0	1	0	0.0
8	14935	1	935	297	1784.71	1784.71	10	0.0	1	0	0.0
9	14241	1	146	183	213.7	213.7	10	0.0	1	0	0.0
10	15733	1	62	282	162.3	162.3	10	0.0	1	0	0.0
11	13108	1	298	372	350.06	350.06	10	0.0	1	0	0.0
12	15667	1	240	39	301.32	301.32	13	0.0	1	0	0.0
13	15607	1	202	336	104.76	104.76	13	0.0	1	0	0.0
14	17128	1	76	334	157.09	157.09	14	0.0	1	0	0.0
15	12489	1	103	336	206.93	206.93	14	0.0	1	0	0.0
16	13419	1	275	63	221.06	221.06	16	0.0	1	0	0.0
17	14476	1	110	257	193.0	193.0	17	0.0	1	0	0.0
18	17855	1	197	372	208.97	208.97	17	0.0	1	0	0.0

- 다양한 컬럼들을 활용하여 고객의 구매 패턴과 선호도를 보다 심층적으로 이해할 수 있도록 최종적으로 `user_data`를 출력하기

```

SELECT *
FROM `modulabs_project.user_data`

```

일	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	average_item	total_transactions	cancel_frequency	cancel_rate
1	17986	1	10	56	20.8	20.8	1	0.0	1	0	0.0
2	17331	1	16	123	175.2	175.2	1	0.0	1	0	0.0
3	16953	1	10	30	20.8	20.8	1	0.0	1	0	0.0
4	16162	1	4	252	37.4	37.4	2	0.0	1	0	0.0
5	13967	1	42	145	80.7	80.7	2	0.0	1	0	0.0
6	14816	1	5	197	271.85	271.85	4	0.0	1	0	0.0
7	18191	1	140	261	207.8	207.8	7	0.0	1	0	0.0

회고

[회고 내용을 작성해주세요]

Keep :

1. LMS에서 원하는 대로 쿼리를 어느 정도 작성할 수 있었음

Problem :

1. CREATE 같은 DDL에 대해서 매우 고난을 겪었음. (중복 컬럼 에러)
2. 다시 되돌리기가 매우 힘들어서 복기하는데 시간을 쏟았음

Try :

1. 되돌리기 힘들다는 걸 알았기 때문에 처음에 잘 작성하기..
2. 더 힘내보기..